College of Computer Science and Information Technology
Department of Computer Engineering

ARTI403: Image Processing

Lab 5: Spatial Filtering: Smoothing and Sharpening

Student Name: Saad Abdulaziz Alshehri
Student ID: 2240006495

Objective

To write a program that implements fundamental image processing algorithms.

Tools

Python 3, Jupyter, OpenCV, NumPy, Matplotlib, scikit-image.

Procedural Steps

Task 1: Unsharp Masking

I used the Chelsea image from scikit-image (the Parrot image from the manual is not available to me). I blurred the image with a Gaussian filter (sigma = 2), then added the difference between the original and the blurred image back to the original with amount = 1.5.

```
blurred = cv2.GaussianBlur(image_float, (0, 0), sigmaX=2, sigmaY=2)
sharpened = np.clip(image_float + 1.5 * (image_float - blurred), 0, 1)
```



Figure 1: Original, blurred and sharpened images

The blurred image lost the fine details, and the sharpened image has clearer edges and fur texture than the original.

Assessment

Task 1: 7x7 box filter

I made a 7x7 kernel where every value is 1/49 and convolved it with the Chelsea image using *cv2.filter2D*.

```
box_kernel = np.ones((7, 7), np.float32) / (7 * 7)
smoothed = cv2.filter2D(image, -1, box_kernel)
```



Figure 2: Original and smoothed image side by side

Each pixel became the average of its 7x7 neighborhood, so the image looks smooth and the small details are gone.

Task 2: 5x5 and 21x21 Gaussian filters

I chose the astronaut image. I applied a 5x5 Gaussian filter and a 21x21 Gaussian filter and showed them next to the original.

```
gauss_5 = cv2.GaussianBlur(image, (5, 5), 0)
gauss_21 = cv2.GaussianBlur(image, (21, 21), 0)
```



Figure 3: Original, Gaussian 5x5 and Gaussian 21x21

The 5x5 filter blurs the image a little. The 21x21 filter blurs it a lot, so a bigger kernel gives a stronger blur.

Task 3: 3x3 Laplacian sharpening

I used the astronaut image again. I read the image, converted it to float32, applied *cv2.Laplacian* with a 3x3 kernel, and subtracted the result from the image.

```
image_float = image.astype(np.float32) / 255.0
laplacian = cv2.Laplacian(image_float, cv2.CV_32F, ksize=1)
sharpened = np.clip(image_float - laplacian, 0, 1)
```

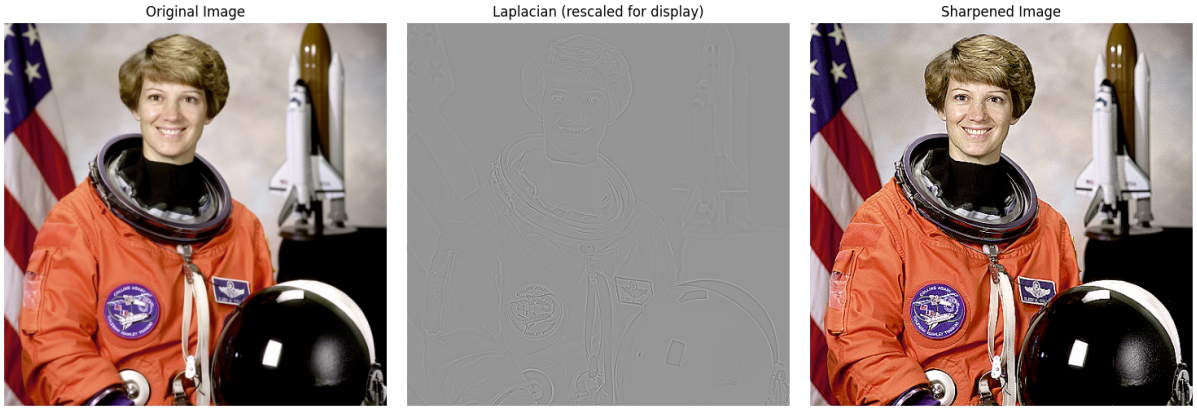


Figure 4: Original, Laplacian (rescaled for display) and sharpened image

The Laplacian image shows only the edges. I rescaled it to 0 to 1 just for display because it has negative values. Subtracting it from the original made the edges stronger.